Travaux pratique n°4 : Installation et configuration d’un serveur web.

5 JANVIER 2026

Rédiger par : NANA Brandon

Introduction

Dans le cadre d'un projet de mise en œuvre et de sécurisation d'un serveur web, une machine Ubuntu 22.04 (ou supérieure) a été déployée afin d'héberger un site via Apache2 et d'assurer sa disponibilité en environnement de test.

L'objectif est d'appliquer une démarche progressive de durcissement : configuration du service web (VirtualHost), réduction des informations exposées par Apache, restriction d'accès à des zones sensibles, puis activation du chiffrement HTTPS à l'aide d'un certificat auto-signé généré avec OpenSSL.

Une phase de sécurisation avancée complète l'installation par l'ajout d'un WAF (ModSecurity + règles OWASP CRS), la protection contre les attaques par brute-force via Fail2Ban, la mise en place d'en-têtes HTTP recommandés par l'OWASP, ainsi qu'un monitoring des accès (GoAccess) et une validation par des tests de sécurité (Nikto, testssl.sh, vérifications curl).

Sommaire

CONTEXTE, OBJECTIFS, PERIMETRE 5

1. CONTEXTE 5
2. OBJECTIFS 5
3. PERIMETRE 6

PRESENTATION DE L'ENVIRONNEMENT (VM/OS, PREREQUIS, RESEAU, COMPTES, ET ACCES) 7

PLATEFORME ET PREREQUIS 7

SERVICES ET COMPOSANTS DEPLOYES 7

RESEAU, ACCES ET JOURNAUX 7

INSTALLATION ET CONFIGURATION D'APACHE2 (PAQUETS, ARBORESCENCE, VIRTUALHOST, SITE) 8

INSTALLATION DES PAQUETS (VOIRE ANNEXES 1 & 2) 8

ARBORESCENCE ET FICHIERS DE CONFIGURATION (ANNEXE 3 & 4 + SCHEMA DE L'INFRASTRUCTURE) 8

PREMIERES MESURES DE SECURISATION D'APACHE (SERVERTOKENS/SERVERSIGNATURE, BONNES PRATIQUES DE CONFIGURATION). (ANNEXE 6 ET 7) 8

REDUCTION D'INFORMATIONS EXPOSEES (SERVERTOKENS / SERVERSIGNATURE) 8

BONNES PRATIQUES DE CONFIGURATION 9

CONTROLE D'ACCES AUX ZONES SENSIBLES (AUTHENTIFICATION BASIC, RESTRICTIONS). (ANNEXE 8 & 9) 9

OBJECTIF 9

MISE EN PLACE (AUTHENTIFICATION BASIC) 9

RESTRICTIONS COMPLEMENTAIRES (ADRESSE IP) 10

EXEMPLE D'USAGE : PROTECTION D'UN RAPPORT DE MONITORING 10

MISE EN PLACE DU CHIFFREMENT HTTPS (CERTIFICAT AUTO-SIGNE OPENSSL, ACTIVATION SSL, CONFIGURATION APACHE). (ANNEXE 10 & 11) 10

GENERATION D'UN CERTIFICAT AUTO-SIGNE (OPENSSL) 10

ACTIVATION SSL COTE APACHE 10

CONFIGURATION DU VIRTUALHOST HTTPS 11

SECURISATION AVANCEE : WAF MODSECURITY + OWASP CRS (INSTALLATION, ACTIVATION, TESTS). (ANNEXE 12 & 13) 11

INSTALLATION ET ACTIVATION DE MODSECURITY 11

DEPLOIEMENT DES REGLES OWASP CRS 11

CONFIGURATION APACHE (CHARGEMENT DES REGLES) 12

TESTS ET VALIDATION 12

PROTECTION ANTI BRUTE-FORCE: FAIL2BAN (JAILS APACHE, FILTRE MODSECURITY, VÉRIFICATIONS). (ANNEXE 14 & 15) 12

ROLE DE FAIL2BAN 12

INSTALLATION ET ACTIVATION DU SERVICE 12

FILTRE DEDIE MODSECURITY (CORRELATION WAF → BANNISSEMENT) 13

VERIFICATIONS ET EXPLOITATION 13

EN-TETES HTTP DE SECURITE (MODULE HEADERS, CONFIGURATION OWASP, TESTS VIA CURL). (ANNEXE 16 ET 17) 13

ACTIVATION DU MODULE HEADERS 13

CONFIGURATION "OWASP-LIKE" DANS APACHE 14

TESTS ET VALIDATION VIA CURL 14

ANALYSE DES ACCES AVEC GOACCESS 14

PROTECTION DU RAPPORT (ACCES RESTREINT) 15

ROTATION ET CONSERVATION DES LOGS (LOGROTATE) 15

TESTS DE VALIDATION (NIKTO, TESTSSL.SH, CONTROLE DES EN-TETES ET PREUVES DE BLOCAGE WAF). (ANNEXE 21, 22, 23 & 24) 15

AUDIT TLS AVEC TESTSSL.SH 16

CONTROLE DES EN-TETES AVEC CURL 16

PREUVES DE BLOCAGE DU WAF (MODSECURITY) 16

CONCLUSION ET AXES D'AMELIORATION (LIMITES DU CERTIFICAT AUTO-SIGNE, DURCISSEMENT COMPLEMENTAIRE, RECOMMANDATIONS). 16

LIMITES DU CERTIFICAT AUTO-SIGNE 17

DURCISSEMENT COMPLEMENTAIRE 17

RECOMMANDATIONS OPERATIONNELLES 17

Contexte, objectifs, périmètre

1. Contexte

Ce projet s’inscrit dans une démarche de déploiement puis de sécurisation progressive d’un serveur web sous Linux, en s’appuyant sur une machine Ubuntu 22.04 (ou supérieure) et le service Apache2 comme socle d’hébergement.

L’installation vise à mettre en place un site web basé sur le mécanisme de VirtualHost, avec une organisation standard des répertoires et des fichiers de configuration d’Apache sous Ubuntu.

Une fois le service fonctionnel, le serveur est durci afin de réduire la surface d’attaque (réduction des informations divulguées par Apache, restrictions d’accès) et d’introduire un chiffrement HTTPS via un certificat auto-signé généré avec OpenSSL.

2. Objectifs

L’objectif principal est de fournir un serveur web opérationnel, capable de publier un site et de répondre de manière fiable aux requêtes HTTP/HTTPS.

Un objectif de sécurité majeur est d'activer HTTPS et de valider le bon fonctionnement de la couche TLS côté serveur (configuration Apache + certificat auto-signé), afin d'assurer la confidentialité et l'intégrité des échanges, même en environnement de test.

Le projet vise également à renforcer la sécurité applicative en mettant en place un WAF ModSecurity et le jeu de règles OWASP CRS, puis à compléter la défense par une protection anti-force-brute via Fail2Ban, configurée pour exploiter les journaux Apache et ModSecurity.

Enfin, l'objectif inclut la mise en conformité avec des bonnes pratiques de sécurité côté HTTP (en-têtes recommandés) et la mise en place d'un monitoring exploitable (GoAccess, rapport HTML protégé, rotation des logs), avec des tests de validation (curl, Nikto, testssl.sh) pour apporter des preuves de configuration.

3. Périmètre

Le périmètre couvre un serveur unique (Ubuntu + Apache2) configuré pour héberger un site web, activer TLS, et appliquer des contrôles de sécurité au niveau du serveur web (WAF, en-têtes HTTP, restrictions d'accès par authentification Basic sur des zones ou ressources).

Sont inclus dans le périmètre : l'installation des composants, leur configuration, l'observabilité via les logs (Apache et ModSecurity), l'automatisation de la réaction aux événements (bannissement Fail2Ban) et la production d'éléments de vérification via des outils de test.

Ne sont pas inclus : une architecture haute disponibilité (cluster/load balancer), une PKI avec certificat émis par une autorité de certification publique, ni l'intégration d'une chaîne CI/CD ou d'une supervision externalisée (SIEM/monitoring centralisé), le projet restant centré sur la mise en œuvre et le durcissement du serveur web lui-même.

Présentation de l’environnement (VM/OS, prérequis, réseau, comptes, et accès)

Plateforme et prérequis

L’environnement de travail repose sur une machine Linux (Ubuntu de préférence), en 64 bits, avec une configuration minimale indiquée de 2 vCPU, 4096 Mo de RAM et 20 Go de stockage.

L’administration système est réalisée avec un compte disposant des droits sudo/root afin d’installer les paquets et de modifier la configuration des services.

Services et composants déployés

Le serveur web est basé sur Apache2, avec une organisation standard Ubuntu (répertoires de configuration sous /etc/apache2 et publication du site sous /var/www/html).
La sécurisation et l’exploitation incluent l’activation de TLS/HTTPS (certificat auto-signé OpenSSL), l’ajout de ModSecurity et des règles OWASP CRS (WAF), la mise en place de Fail2Ban, ainsi que la configuration d’en-têtes HTTP de sécurité via le module headers.

Le monitoring s’appuie sur l’analyse des journaux via GoAccess (génération possible d’un rapport HTML), complété par la rotation des logs, et la validation est effectuée avec des outils de test tels que Nikto, testssl.sh et des contrôles via la commande curl.

Réseau, accès et journaux

Les accès utilisateurs au service web sont réalisés en HTTP/HTTPS (avec tests possibles en local), et l’accès d’administration est effectué depuis une session terminale (commandes système et tests curl).

Les journaux applicatifs (notamment ceux d’Apache et de ModSecurity) constituent la base de la détection et de la réaction automatisée (Fail2Ban) ainsi que des analyses de trafic (GoAccess).

Installation et configuration d’apache2 (paquets, arborescence, VirtualHost, site)

Installation des paquets (Voire annexes 1 & 2)

L’installation d’Apache2 est réalisée via le gestionnaire de paquets APT, après mise à jour de l’index des dépôts (sudo apt update) et éventuellement mise à niveau des paquets (sudo apt upgrade), puis installation du service (sudo apt-get install apache2).
Après installation, le service est administré via systemctl (rechargement/redémarrage) pour appliquer les changements de configuration.

Arborescence et fichiers de configuration (annexe 3 & 4 + Schéma de l’infrastructure)

Sous Ubuntu, la configuration Apache2 est centralisée dans /etc/apache2, notamment avec apache2.conf, ports.conf (directives d’écoute “Listen”) et le fichier envvars pour les variables d’environnement du service.

Le découpage de configuration suit la logique “available/enabled” avec les répertoires mods-available / mods-enabled, conf-available / conf-enabled et sites-available / sites-enabled.
Dans le TP, un ajustement d’écoute est également mentionné (ex. modification de “Listen 80” vers une écoute locale/port alternatif), ce qui illustre la personnalisation des ports d’exposition du service.

Premières mesures de sécurisation d’Apache (ServerTokens/ServerSignature, bonnes pratiques de configuration). (Annexe 6 et 7)

Réduction d’informations exposées (ServerTokens / ServerSignature)

Par défaut, Apache peut renvoyer vers les en-têtes HTTP (et sur certaines pages d’erreur) des informations comme le nom du serveur, sa version, voire des éléments liés à l’OS, ce qui facilite la reconnaissance (fingerprinting) par un attaquant.

Sur Ubuntu, ces directives de durcissement sont regroupées dans /etc/apache2/conf-available/security.conf, où l'on applique typiquement : ServerTokens Prod et ServerSignature Off afin de supprimer les informations non nécessaires.

Bonnes pratiques de configuration

Une bonne pratique consiste à conserver une configuration lisible et modulaire en s'appuyant sur la séparation Ubuntu "available/enabled" (sites, modules, conf), afin de n'activer que ce qui est nécessaire et de faciliter les audits.

Côté VirtualHost, l'approche recommandée est de définir explicitement le DocumentRoot, les fichiers de logs, et des directives de répertoire cohérentes (ex. éviter des permissions trop larges, maîtriser les surcharges via AllowOverride), pour réduire les risques liés à une configuration implicite.

Enfin, contrôler précisément les interfaces/ports d'écoute (via ports.conf / directives Listen) fait partie des actions de durcissement, car cela limite l'exposition du service au strict besoin (ex. écoute locale ou port spécifique en contexte de test).

Contrôle d'accès aux zones sensibles (authentification Basic, restrictions). (annexe 8 & 9)

Objectif

L'objectif est d'éviter qu'un répertoire sensible soit accessible publiquement, même si le site principal reste ouvert, en ajoutant une barrière d'authentification et/ou une limitation réseau. Cette approche réduit le risque d'exposition involontaire (fichiers internes, rapports, pages d'administration) et limite la surface d'attaque.

Mise en place (authentification Basic)

Une zone est d'abord créée dans l'arborescence web (ex. un répertoire /topsecret), puis protégée dans le VirtualHost via un bloc <Directory …>.
Dans ce bloc, Apache est configuré avec AuthType Basic, un message AuthName, un fichier de comptes local (AuthUserFile) et une règle d'accès, typiquement Require valid-user pour

n’autoriser que les utilisateurs authentifiés.
Le fichier de mots de passe est généré avec htpasswd (création du fichier de comptes sous /etc/apache2/...), ce qui permet à Apache de valider les identifiants sans dépendance externe dans le cadre du TP.

Restrictions complémentaires (adresse IP)

Pour renforcer le contrôle d’accès, la configuration peut aussi inclure une restriction par adresse IP (Require ip <IP_autorisée>), afin de limiter l’accès à un poste ou réseau d’administration connu.
Cette restriction peut être utilisée seule ou combinée à l’authentification (approche plus stricte), selon le niveau de sécurité attendu.

Exemple d’usage : protection d’un rapport de monitoring

La même mécanique est applicable pour protéger un rapport HTML (ex. rapport GoAccess généré dans /var/www/html/report.html) afin qu’il ne soit pas accessible à tous.
Le TP décrit explicitement la protection du rapport GoAccess via un bloc <Directory …> ajoutant l’authentification Basic et Require valid-user.

Mise en place du chiffrement HTTPS (certificat auto-signé OpenSSL, activation SSL, configuration Apache). (annexe 10 & 11)

Génération d’un certificat auto-signé (OpenSSL)

Un certificat auto-signé est créé avec OpenSSL en générant simultanément une clé privée et un certificat X.509, avec une durée de validité définie (ex. 365 jours) et une clé RSA 2048 bits.

Le TP illustre cette étape via une commande openssl req -x509 -nodes -days 365 -newkey rsa2048, stockant la clé privée dans /etc/ssl/private/... et le certificat dans /etc/ssl/certs/....

Activation SSL côté Apache

Une fois le certificat disponible, le module SSL d’Apache est activé avec a2enmod ssl, ce qui permet au serveur de gérer des VirtualHosts HTTPS.

Le VirtualHost TLS par défaut peut ensuite être activé via a2ensite default-ssl, puis Apache est redémarré pour appliquer les changements.

Configuration du VirtualHost HTTPS

La configuration HTTPS se fait dans le fichier du site SSL (ex. default-ssl.conf) en remplaçant les chemins par défaut par ceux du certificat auto-signé et de la clé privée (directives SSLCertificateFile et SSLCertificateKeyFile).

Une fois la configuration appliquée, un test d'accès en https://localhost permet de valider le service ; l'avertissement "certificat non reconnu / auto-signé" est attendu côté client, et les tests en ligne de commande peuvent nécessiter l'option -k avec curl pour ignorer la validation du certificat.

Sécurisation avancée : WAF ModSecurity + OWASP CRS (installation, activation, tests). (Annexe 12 & 13)

Installation et activation de ModSecurity

ModSecurity est installé via le paquet libapache2-mod-security2, puis activé au niveau d'Apache avec a2enmod security2, suivi d'un redémarrage du service pour charger le module. Une configuration de base est appliquée en copiant le fichier recommandé (modsecurity.conf-recommended vers modsecurity.conf) puis en passant le moteur de règles de DetectionOnly à On afin de ne plus seulement détecter mais également bloquer.

Déploiement des règles OWASP CRS

Le projet s'appuie ensuite sur l'OWASP Core Rule Set (CRS), qui fournit des règles standardisées pour la détection et la mitigation d'attaques web.
Le TP propose l'installation du paquet modsecurity-crs et/ou l'obtention des règles via un dépôt (git clone), puis la mise en place des fichiers crs-setup.conf et du répertoire rules dans l'arborescence ModSecurity.

Configuration Apache (chargement des règles)

Pour que ModSecurity et les règles CRS soient bien prises en compte, la configuration du module Apache (security2.conf) doit inclure les fichiers de configuration ModSecurity et le fichier de règles (ex. IncludeOptional /etc/modsecurity/modsecurity.conf et IncludeOptional /etc/modsecurity/rules.conf).
Une fois ces inclusions en place, un redémarrage d'Apache est effectué pour activer le WAF dans la chaîne de traitement des requêtes.

Tests et validation

La validation est réalisée en envoyant une requête volontairement malveillante (ex. XSS simulé) avec curl afin de vérifier que ModSecurity détecte et bloque, puis en consultant les traces dans le journal d'audit (ex. modsec_audit.log).
Les captures issues du TP montrent justement des événements ModSecurity dans les logs, avec des règles CRS déclenchées et un blocage (code 403) lors d'un test XSS.

Protection anti brute-force: Fail2Ban (jails Apache, filtre ModSecurity, vérifications). (Annexe 14 & 15)

Rôle de Fail2Ban

Fail2Ban analyse des fichiers de logs et applique des bannissements temporaires lorsqu'un seuil de tentatives est dépassé, ce qui réduit l'efficacité des attaques de type brute force et scanning automatisé.
Dans ce projet, Fail2Ban est utilisé comme couche défensive complémentaire au WAF : ModSecurity bloque/détecte des attaques web, tandis que Fail2Ban permet de sanctionner la source (IP) en cas de répétition.

Installation et activation du service

Fail2Ban est installé via le paquet fail2ban, puis activé au démarrage et démarré via systemctl enable fail2ban et systemctl restart fail2ban.

Le fonctionnement est ensuite vérifié avec fail2ban-client status, qui permet de confirmer que le service tourne et que les “jails” sont chargés.

Configuration des jails Apache

La configuration est définie dans un fichier local jail.local, en précisant des paramètres globaux tels que bantime, findtime, maxretry ainsi qu’une liste d’IP à exclure (ignoreip).
Plusieurs jails orientés Apache sont activés, notamment apache-auth (tentatives d’authentification), apache-badbots (user-agents/robots malveillants), apache-noscript et apache-overflows, en pointant explicitement vers les bons fichiers de logs (/var/log/apache2/error.log et /var/log/apache2/access.log) et les ports http, https.

Filtre dédié ModSecurity (corrélation WAF → bannissement)

Un jail spécifique apache-modsecurity est configuré pour exploiter le journal d’audit ModSecurity (/var/log/apache2/modsec_audit.log), afin de bannir les IP qui déclenchent des blocages WAF de manière répétée.
Pour cela, un filtre dédié est créé (ex. filter.d/apache-modsecurity.conf) avec une expression de type failregex permettant de repérer dans les logs les événements ModSecurity associés à une IP cliente.

Vérifications et exploitation

La vérification opérationnelle se fait avec fail2ban-client status (liste des jails actifs) et fail2ban-client status <jail> (détail d’un jail, IP bannies, compteurs).
Le TP mentionne également la possibilité de débannir manuellement une IP (ex. fail2ban-client set apache-auth unbanip <IP>), utile en cas de faux positifs pendant les tests.

En-têtes HTTP de sécurité (module headers, configuration OWASP, tests via curl). (Annexe 16 et 17)

Activation du module headers

La première étape consiste à activer le module Apache dédié aux en-têtes avec a2enmod headers, puis à redémarrer Apache pour prendre en compte l’activation.

Ce module permet d'ajouter/surcharger des en-têtes HTTP sur toutes les réponses afin d'appliquer une politique de sécurité cohérente.

Configuration "OWASP-like" dans Apache

La configuration est réalisée dans le fichier de durcissement d'Apache (sur Ubuntu : /etc/apache2/conf-available/security.conf) en ajoutant une série d'en-têtes recommandés.

Le TP met notamment en place : X-Frame-Options, X-XSS-Protection, X-Content-Type-Options, Referrer-Policy, Content-Security-Policy, Permissions-Policy et Strict-Transport-Security (HSTS).

Après ajout/modification, Apache est redémarré afin d'appliquer la nouvelle politique d'en-têtes sur les réponses HTTP/HTTPS.

Tests et validation via curl

La validation se fait en interrogeant le serveur et en affichant les en-têtes retournés, typiquement avec curl -I (et -k en HTTPS si le certificat est auto-signé).

Le TP fournit un exemple d'en-têtes visibles dans une réponse HTTP 200 (ex. présence de X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Content-Security-Policy, Permissions-Policy).

Un contrôle ciblé peut aussi être réalisé en filtrant un en-tête précis (ex. grep -i content-security-policy) pour prouver la présence de la CSP.

Monitoring et exploitation des logs (GoAccess, protection du rapport, logrotate) (annexe 18, 19 & 20)

Analyse des accès avec GoAccess

GoAccess est installé puis utilisé comme analyseur de logs "temps réel" afin d'obtenir des statistiques détaillées à partir du fichier /var/log/apache2/access.log.

Le TP prévoit une utilisation interactive (goaccess -c) et la génération d'un rapport HTML (ex. /var/www/html/report.html) en précisant le format de logs log-format=COMBINED, avec une option possible de rafraîchissement temps réel.

Ce rapport facilite l'exploitation opérationnelle (top URL, codes HTTP, visiteurs, volumes) sans devoir parser les fichiers à la main.

Protection du rapport (accès restreint)

Comme le rapport GoAccess peut révéler des informations sensibles (URLs, structure applicative, volumes, comportements), il est protégé par une authentification Basic dans la configuration du site Apache.
Le TP décrit l'ajout d'un bloc de protection (type <Directory >) avec AuthType Basic, AuthUserFile et Require valid-user pour restreindre l'accès au rapport HTML.

Rotation et conservation des logs (logrotate)

Pour éviter que les journaux ne saturent le disque et pour conserver un historique exploitable, la rotation est configurée via logrotate (fichier de configuration Apache2).

Le TP propose une rotation quotidienne, une rétention (ex. rotate 14), la compression, et un postrotate déclenchant un rechargement d'Apache afin de rouvrir proprement les fichiers de logs.

Tests de validation (Nikto, testssl.sh, contrôle des en-têtes et preuves de blocage WAF). (annexe 21, 22, 23 & 24)

Scan applicatif avec Nikto

Nikto est utilisé comme scanner de vulnérabilités web afin d'identifier des faiblesses courantes côté serveur et de vérifier qu'aucune configuration "évidente" n'est exposée.
Le TP prévoit l'exécution d'un scan sur l'URL HTTPS (ex. nikto -h https://localhost -ssl) et peut produire un rapport HTML via -o -Format html.
Les résultats permettent notamment de confirmer la présence d'en-têtes de sécurité et de relever d'éventuels points d'amélioration.

Audit TLS avec testssl.sh

testssl.sh est utilisé pour analyser la configuration SSL/TLS (protocoles supportés, chiffrements, vulnérabilités connues, compatibilités client).
Le TP décrit un lancement sur l'URL HTTPS et la génération possible d'un rapport HTML.
Cela fournit une preuve structurée du niveau TLS réellement proposé par le serveur.

Contrôle des en-têtes avec curl

Les en-têtes de sécurité sont contrôlés via curl -I (et -k en cas de certificat auto-signé) afin de vérifier que la réponse HTTP contient bien les en-têtes configurés (ex. CSP, Referrer-Policy, X-Frame-Options, etc.).
Le TP propose aussi des vérifications ciblées d'un en-tête spécifique (ex. filtrage sur content-security-policy).

Preuves de blocage du WAF (ModSecurity)

Pour démontrer l'efficacité du WAF, une requête volontairement malveillante est envoyée (ex. payload XSS ou injection) via curl, puis on vérifie le résultat côté serveur (blocage HTTP) et côté logs (événement dans modsec_audit.log).
Le TP illustre ce contrôle par des tests XSS/injection SQL et la consultation des dernières lignes du journal d'audit ModSecurity, montrant des règles CRS déclenchées et un blocage (code 403).

Conclusion et axes d'amélioration (limites du certificat auto-signé, durcissement complémentaire, recommandations).

Les contrôles réalisés valident que le serveur applique une défense en profondeur : filtrage applicatif, blocage/ban des comportements abusifs, renforcement côté navigateur et exploitation centralisée des logs.
Cette approche réduit l'impact des scans automatisés et améliore la détectabilité des tentatives d'intrusion via l'analyse des journaux et des alertes associées

Limites du certificat auto-signé

Un certificat auto-signé provoque des alertes côté client et n’établit pas une confiance “publique” : le navigateur/curl ne peut pas vérifier la légitimité du serveur sans ajout manuel du certificat dans le trust store.
Dans le TP, le HTTPS fonctionne mais la validation échoue sans option de contournement (ex. -k avec curl), ce qui n’est pas acceptable en production.

Durcissement complémentaire

Remplacer le certificat auto-signé par un certificat émis par une AC reconnue (ex. Let’s Encrypt) et automatiser le renouvellement afin d’éviter l’expiration et les interruptions de service.
Réduire la surface d’attaque Apache : désactiver modules inutiles, limiter les méthodes HTTP, appliquer le principe du moindre privilège sur les répertoires, et cloisonner l’application (compte dédié, permissions minimales).
Ajuster finement le WAF (seuils, exclusions, mode paranoïa) pour limiter les faux positifs, tout en renforçant la journalisation et la traçabilité des blocages.

Recommandations opérationnelles

Mettre en place une stratégie de supervision : tableaux de bord (trafic/erreurs), alertes (pics 4xx/5xx, bans Fail2Ban), et rotation/conservation des logs avec une rétention adaptée aux besoins d’audit.
Documenter une procédure de réponse à incident (triage, extraction des preuves dans les logs, rollback de configuration) et planifier des scans récurrents (web + TLS) après chaque changement de configuration.

Annexes

Schéma de l’infrastructure (Voire l’image inclus dans les rendus)

Annexe 1

```
brandon@brandon:~$ apache2 -v
Server version: Apache/2.4.58 (Ubuntu)
Server built:   2025-08-11T11:10:09
```

Annexe 2

```
brandon@brandon:~$ systemctl status apache2
● apache2.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: >
     Active: active (running) since Tue 2025-12-30 14:20:54 CET; 31s ago
       Docs: https://httpd.apache.org/docs/2.4/
    Process: 1709 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SU>
   Main PID: 1793 (apache2)
      Tasks: 55 (limit: 4542)
     Memory: 56.1M (peak: 56.5M)
        CPU: 791ms
     CGroup: /system.slice/apache2.service
             ├─1793 /usr/sbin/apache2 -k start
             ├─1812 /usr/sbin/apache2 -k start
             └─1813 /usr/sbin/apache2 -k start

déc. 30 14:20:53 brandon systemd[1]: Starting apache2.service - The Apache HTTP>
déc. 30 14:20:54 brandon apachectl[1745]: AH00558: apache2: Could not reliably >
déc. 30 14:20:54 brandon systemd[1]: Started apache2.service - The Apache HTTP >
lines 1-17/17 (END)
```

Annexe 3

```
brandon@brandon:~$ tree -L 2 /etc/apache2
/etc/apache2
├── apache2.conf
├── conf-available
│   ├── charset.conf
│   ├── localized-error-pages.conf
│   ├── other-vhosts-access-log.conf
│   ├── security.conf
│   └── serve-cgi-bin.conf
├── conf-enabled
│   ├── charset.conf -> ../conf-available/charset.conf
│   ├── localized-error-pages.conf -> ../conf-available/localized-error-pages.conf
│   ├── other-vhosts-access-log.conf -> ../conf-available/other-vhosts-access-log.conf
│   ├── security.conf -> ../conf-available/security.conf
│   └── serve-cgi-bin.conf -> ../conf-available/serve-cgi-bin.conf
├── envvars
├── magic
├── mods-available
│   ├── access_compat.load
│   ├── actions.conf
│   ├── actions.load
│   ├── alias.conf
│   ├── alias.load
│   ├── allowmethods.load
│   ├── asis.load
│   ├── auth_basic.load
│   ├── auth_digest.load
│   ├── auth_form.load
│   ├── authn_anon.load
│   ├── authn_core.load
│   ├── authn_dbd.load
│   ├── authn_dbm.load
│   ├── authn_file.load
│   ├── authn_socache.load
│   ├── authnz_fcgi.load
│   ├── authnz_ldap.load
│   ├── authz_core.load
│   ├── authz_dbd.load
│   ├── authz_dbm.load
│   ├── authz_groupfile.load
│   ├── authz_host.load
│   ├── authz_owner.load
│   ├── authz_user.load
│   ├── autoindex.conf
│   ├── autoindex.load
```

```
brandon@brandon:~$ ls -la /etc/apache2/sites-available
total 32
drwxr-xr-x 2 root root 4096 déc.  27 21:06 .
drwxr-xr-x 8 root root 4096 déc.  26 14:41 ..
-rw-r--r-- 1 root root 1286 mars  18  2024 000-default.conf
-rw-r--r-- 1 root root 4581 déc.  27 21:06 default-ssl.conf
-rw-r--r-- 1 root root  144 déc.  27 20:31 localhost.conf
-rw-r--r-- 1 root root  224 déc.  26 15:34 localhost-ssl.conf
-rw-r--r-- 1 root root  166 déc.  26 15:55 www.mysite.net.conf
brandon@brandon:~$
```

```
brandon@brandon:~$ ls -la /etc/apache2/sites-enabled
total 8
drwxr-xr-x 2 root root 4096 déc.  27 21:01 .
drwxr-xr-x 8 root root 4096 déc.  26 14:41 ..
lrwxrwxrwx 1 root root   35 déc.  27 21:01 default-ssl.conf -> ../sites-available/default-ssl.conf
lrwxrwxrwx 1 root root   33 déc.  26 15:39 localhost.conf -> ../sites-available/localhost.conf
lrwxrwxrwx 1 root root   37 déc.  26 15:34 localhost-ssl.conf -> ../sites-available/localhost-ssl.conf
brandon@brandon:~$
```

Annexe 4

```
26 déc. 15:24
brandon@brandon: /etc/modsecurity
  GNU nano 7.2          /etc/apache2/conf-available/security.conf *
#RedirectMatch 404 /\.svn

#
# Setting this header will prevent MSIE from interpreting files as something
# else than declared by the content type in the HTTP headers.
# Requires mod_headers to be enabled.
#
#Header set X-Content-Type-Options: "nosniff"

#
# Setting this header will prevent other sites from embedding pages from this
# site as frames. This defends against clickjacking attacks.
# Requires mod_headers to be enabled.
#
#Header set Content-Security-Policy "frame-ancestors 'self';"
# Protection contre le clickjacking
Header always set X-Frame-Options "SAMEORIGIN"
# Protection XSS (navigateurs anciens)
Header always set X-XSS-Protection "1; mode=block"
# Empêcher le MIME-sniffing
Header always set X-Content-Type-Options "nosniff"
# Politique de référent
Header always set Referrer-Policy "strict-origin-when-cross-origin"
# Content Security Policy (CSP)
Header always set Content-Security-Policy "default-src 'self'; \
script-src 'self'; style-src 'self' 'unsafe-inline'; \
img-src 'self' data:; font-src 'self'; \
frame-ancestors 'self'; form-action 'self'"
# Permissions Policy (remplace Feature-Policy)
Header always set Permissions-Policy "geolocation=(), microphone=(), camera=()"
# Strict Transport Security (HSTS) - uniquement si HTTPS Header always set StrictTrans>
"max-age=31536000; includeSubDomains; preload"
```

```
brandon@brandon:~$ ls -la /etc/apache2/sites-available
total 32
drwxr-xr-x 2 root root 4096 déc.  27 21:06 .
drwxr-xr-x 8 root root 4096 déc.  26 14:41 ..
-rw-r--r-- 1 root root 1286 mars  18  2024 000-default.conf
-rw-r--r-- 1 root root 4581 déc.  27 21:06 default-ssl.conf
-rw-r--r-- 1 root root  144 déc.  27 20:31 localhost.conf
-rw-r--r-- 1 root root  224 déc.  26 15:34 localhost-ssl.conf
-rw-r--r-- 1 root root  166 déc.  26 15:55 www.mysite.net.conf
brandon@brandon:~$
```

```
brandon@brandon: /etc/modsecurity
Fichier  Édition  Affichage  Rechercher  Terminal  Aide
  GNU nano 7.2          /etc/apache2/sites-available/localhost.conf
<VirtualHost *:80>
  ServerName localhost
  Redirect / https://localhost/
</VirtualHost>

                          [ Lecture de 4 lignes ]
^G Aide      ^O Écrire    ^W Chercher  ^K Couper    ^T Exécuter  ^C Emplacement
^X Quitter   ^R Lire fich. ^\ Remplacer ^U Coller    ^J Justifier ^/ Aller ligne
```

Annexe 5

```
brandon@brandon: ~
  GNU nano 7.2    /etc/apache2/sites-available/www.mysite.net.conf
<Directory /var/www/html/report.html>
 AuthType Basic
 AuthName "Monitoring - Accès restreint"
 AuthUserFile /etc/apache2/passwords
 Require valid-user
</Directory>

                          [ Lecture de 6 lignes ]
^G Aide      ^O Écrire    ^W Chercher  ^K Couper    ^T Exécuter  ^C Emplacement
^X Quitter   ^R Lire fich.^\ Remplacer  ^U Coller    ^J Justifier ^/ Aller ligne
```

Annexe 6

```
brandon@brandon: ~
  GNU nano 7.2                                          /etc/apache2/conf-available/security.conf
# Changing the following options will not really affect the security of the
# server, but might make attacks slightly more difficult in some cases.

#
# ServerTokens
# This directive configures what you return as the Server HTTP response
# Header. The default is 'Full' which sends information about the OS-Type
# and compiled in modules.
# Set to one of:  Full | OS | Minimal | Minor | Major | Prod
# where Full conveys the most information, and Prod the least.
#ServerTokens Minimal
ServerTokens Prod
#ServerTokens Full

#
# Optionally add a line containing the server version and virtual host
# name to server-generated pages (internal error documents, FTP directory
# listings, mod_status and mod_info output etc., but not CGI generated
# documents or custom error documents).
# Set to "EMail" to also include a mailto: link to the ServerAdmin.
# Set to one of:  On | Off | EMail
#ServerSignature Off
ServerSignature Off

#
# Allow TRACE method
#
# Set to "extended" to also reflect the request body (only for testing and
# diagnostic purposes).
#
# Set to one of:  On | Off | extended
TraceEnable Off
#TraceEnable On

#
# Forbid access to version control directories
#
# If you use version control systems in your document root, you should
# probably deny access to their directories.
#
# Examples:
                                                                          [ 75 lignes écrites ]
```

```
brandon@brandon:~$ curl -I https://192.168.142.135
curl: (60) SSL: no alternative certificate subject name matches target host name '192.168.142.135'
More details here: https://curl.se/docs/sslcerts.html

curl failed to verify the legitimacy of the server and therefore could not
establish a secure connection to it. To learn more about this situation and
how to fix it, please visit the web page mentioned above.
```

Annexe 7

```
26 déc. 15:24
brandon@brandon: /etc/modsecurity
  GNU nano 7.2          /etc/apache2/conf-available/security.conf *
#RedirectMatch 404 /\.svn

#
# Setting this header will prevent MSIE from interpreting files as something
# else than declared by the content type in the HTTP headers.
# Requires mod_headers to be enabled.
#
#Header set X-Content-Type-Options: "nosniff"

#
# Setting this header will prevent other sites from embedding pages from this
# site as frames. This defends against clickjacking attacks.
# Requires mod_headers to be enabled.
#
#Header set Content-Security-Policy "frame-ancestors 'self';"
# Protection contre le clickjacking
Header always set X-Frame-Options "SAMEORIGIN"
# Protection XSS (navigateurs anciens)
Header always set X-XSS-Protection "1; mode=block"
# Empêcher le MIME-sniffing
Header always set X-Content-Type-Options "nosniff"
# Politique de référent
Header always set Referrer-Policy "strict-origin-when-cross-origin"
# Content Security Policy (CSP)
Header always set Content-Security-Policy "default-src 'self'; \
script-src 'self'; style-src 'self' 'unsafe-inline'; \
img-src 'self' data:; font-src 'self'; \
frame-ancestors 'self'; form-action 'self'"
# Permissions Policy (remplace Feature-Policy)
Header always set Permissions-Policy "geolocation=(), microphone=(), camera=()"
# Strict Transport Security (HSTS) - uniquement si HTTPS Header always set StrictTrans>
"max-age=31536000; includeSubDomains; preload"
```

Annexe 8

```
brandon@brandon: /etc/modsecurity
Fichier  Édition  Affichage  Rechercher  Terminal  Aide
  GNU nano 7.2          /etc/apache2/sites-available/localhost.conf
<VirtualHost *:80>
  ServerName localhost
  Redirect / https://localhost/
</VirtualHost>

                          [ Lecture de 4 lignes ]
^G Aide      ^O Écrire    ^W Chercher  ^K Couper    ^T Exécuter  ^C Emplacement
^X Quitter   ^R Lire fich. ^\ Remplacer ^U Coller    ^J Justifier ^/ Aller ligne
```

```
brandon@brandon: ~
  GNU nano 7.2     /etc/apache2/sites-available/www.mysite.net.conf
<Directory /var/www/html/report.html>
 AuthType Basic
 AuthName "Monitoring - Accès restreint"
 AuthUserFile /etc/apache2/passwords
 Require valid-user
</Directory>

                              [ Lecture de 6 lignes ]
^G Aide      ^O Écrire    ^W Chercher  ^K Couper    ^T Exécuter  ^C Emplacement
^X Quitter   ^R Lire fich.^\ Remplacer ^U Coller    ^J Justifier ^/ Aller ligne
```

Annexe 9

```
brandon@brandon: /etc/modsecurity
Fichier  Édition  Affichage  Rechercher  Terminal  Aide
brandon@brandon:/etc/modsecurity$ curl -I https://localhost
curl: (60) SSL certificate problem: self-signed certificate
More details here: https://curl.se/docs/sslcerts.html

curl failed to verify the legitimacy of the server and therefore could not
establish a secure connection to it. To learn more about this situation and
how to fix it, please visit the web page mentioned above.
brandon@brandon:/etc/modsecurity$ curl -k -I https://localhost
HTTP/1.1 200 OK
Date: Fri, 26 Dec 2025 14:42:35 GMT
Server: Apache/2.4.58 (Ubuntu)
X-Frame-Options: SAMEORIGIN
X-XSS-Protection: 1; mode=block
X-Content-Type-Options: nosniff
Referrer-Policy: strict-origin-when-cross-origin
Content-Security-Policy: default-src 'self'; script-src 'self'; style-src 'self' 'unsaf
e-inline'; img-src 'self' data:; font-src 'self'; frame-ancestors 'self'; form-action '
self'
Permissions-Policy: geolocation=(), microphone=(), camera=()
Last-Modified: Fri, 26 Dec 2025 13:41:38 GMT
ETag: "29af-646db0d85d3fd"
Accept-Ranges: bytes
Content-Length: 10671
Vary: Accept-Encoding
Content-Type: text/html

brandon@brandon:/etc/modsecurity$
```

Annexe 10

```
brandon@brandon:/etc/modsecurity$ sudo openssl req -x509 -nodes -days 365 -newkey rsa:2
048 \
  -keyout /etc/ssl/private/apache-selfsigned.key \
  -out /etc/ssl/certs/apache-selfsigned.crt
....+.....+...+..+....+...+...........+....+..+......+...+....+..+.+.........+.....+.
..+.......+.......+......+....+..+++++++++++++++++++++++++++++++++++++++++++++++++++++
++++++++++++*.........+...............+...+..........+..+......+...+.........+........
......+...+...+....+...+++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
+*..+......+.+.....+.+...........+........+...+....+...........+....+..+...+.+........
.+.....+...+..................+.+...+...+........+......+.+........+....+..+...........
........+...........+...+.+..+....+.....+..........+.........+..+...................+..
....+..+.+..+...+....+...........+....+...........+....+...+........+........+.......
..+........+........+.+..+.......+......++++++++++++++++++++++++++++++++++++++++++++++
+++++++++++++++++++
......+......+.....+......+.......+...+..+.+.....+++++++++++++++++++++++++++++++++++++
++++++++++++++++++++++++++*....+........+...+....+...+..+.+...+.....+.+..............+
.......+........+....+..+.+............+..+.++++++++++++++++++++++++++++++++++++++++++
++++++++++++++++++++++*.+...+.++++++++++++++++++++++++++++++++++++++++++++++++++++++++
++++++++
-----
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:FR
State or Province Name (full name) [Some-State]:Paris
Locality Name (eg, city) []:Chantilly
Organization Name (eg, company) [Internet Widgits Pty Ltd]:Efrei
Organizational Unit Name (eg, section) []:Security
Common Name (e.g. server FQDN or YOUR name) []:localhost
Email Address []:
brandon@brandon:/etc/modsecurity$
```

```
brandon@brandon:/etc/modsecurity$ sudo nano /etc/apache2/sites-available/localhost-ssl.
conf :
brandon@brandon:/etc/modsecurity$ sudo nano /etc/apache2/sites-available/localhost-ssl.
conf :
brandon@brandon:/etc/modsecurity$ sudo a2ensite localhost-ssl.conf
Enabling site localhost-ssl.
To activate the new configuration, you need to run:
  systemctl reload apache2
brandon@brandon:/etc/modsecurity$ sudo apache2ctl configtest
AH00558: apache2: Could not reliably determine the server's fully qualified domain name
, using 127.0.1.1. Set the 'ServerName' directive globally to suppress this message
Syntax OK
brandon@brandon:/etc/modsecurity$ sudo systemctl reload apache2
brandon@brandon:/etc/modsecurity$
```

Annexe 11

```
brandon@brandon: /etc/modsecurity
Fichier  Édition  Affichage  Rechercher  Terminal  Aide
brandon@brandon:/etc/modsecurity$ curl -I https://localhost
curl: (60) SSL certificate problem: self-signed certificate
More details here: https://curl.se/docs/sslcerts.html

curl failed to verify the legitimacy of the server and therefore could not
establish a secure connection to it. To learn more about this situation and
how to fix it, please visit the web page mentioned above.
brandon@brandon:/etc/modsecurity$ curl -k -I https://localhost
HTTP/1.1 200 OK
Date: Fri, 26 Dec 2025 14:42:35 GMT
Server: Apache/2.4.58 (Ubuntu)
X-Frame-Options: SAMEORIGIN
X-XSS-Protection: 1; mode=block
X-Content-Type-Options: nosniff
Referrer-Policy: strict-origin-when-cross-origin
Content-Security-Policy: default-src 'self'; script-src 'self'; style-src 'self' 'unsaf
e-inline'; img-src 'self' data:; font-src 'self'; frame-ancestors 'self'; form-action '
self'
Permissions-Policy: geolocation=(), microphone=(), camera=()
Last-Modified: Fri, 26 Dec 2025 13:41:38 GMT
ETag: "29af-646db0d85d3fd"
Accept-Ranges: bytes
Content-Length: 10671
Vary: Accept-Encoding
Content-Type: text/html

brandon@brandon:/etc/modsecurity$
```

Annexe 12

```
brandon@brandon: ~
  GNU nano 7.2                                  /etc/apache2/mods-enabled/security2.conf
<IfModule security2_module>
SecDataDir /var/cache/modsecurity
IncludeOptional /etc/modsecurity/*.conf
IncludeOptional /etc/modsecurity/rules/*.conf

</IfModule>
```

```
brandon@brandon:/etc/modsecurity$ curl 'http://localhost/?param=<script>alert(1)
</script>'
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>403 Forbidden</title>
</head><body>
<h1>Forbidden</h1>
<p>You don't have permission to access this resource.</p>
<hr>
<address>Apache/2.4.58 (Ubuntu) Server at localhost Port 80</address>
</body></html>
brandon@brandon:/etc/modsecurity$
```

```
brandon@brandon:/etc/modsecurity$ sudo tail -f /var/log/apache2/modsec_audit.log
Action: Intercepted (phase 2)
Stopwatch: 1766757648348968 3331 (- - -)
Stopwatch2: 1766757648348968 3331; combined=2778, p1=1102, p2=1528, p3=0, p4=0,
p5=148, sr=0, sw=0, l=0, gc=0
Response-Body-Transformed: Dechunked
Producer: ModSecurity for Apache/2.9.7 (http://www.modsecurity.org/); OWASP_CRS/
4.22.0-dev.
Server: Apache/2.4.58 (Ubuntu)
Engine-Mode: "ENABLED"

--dc6a4b58-Z--
```

Annexe 13

```
brandon@brandon: ~
  GNU nano 7.2                                        /etc/modsecurity/modsecurity.conf
# -- Rule engine initialization ----------------------------------------------

# Enable ModSecurity, attaching it to every transaction. Use detection
# only to start with, because that minimises the chances of post-installation
# disruption.
#
SecRuleEngine On


# -- Request body handling ---------------------------------------------------

# Allow ModSecurity to access request bodies. If you don't, ModSecurity
# won't be able to see any POST parameters, which opens a large security
# hole for attackers to exploit.
#
SecRequestBodyAccess On


# Enable XML request body parser.
# Initiate XML Processor in case of xml content-type
#
SecRule REQUEST_HEADERS:Content-Type "^(?:application(?:/soap\+|/)|text/)xml" \
     "id:'200000',phase:1,t:none,t:lowercase,pass,nolog,ctl:requestBodyProcessor=XML"

# Enable JSON request body parser.
# Initiate JSON Processor in case of JSON content-type; change accordingly
# if your application does not use 'application/json'
#
SecRule REQUEST_HEADERS:Content-Type "^application/json" \
     "id:'200001',phase:1,t:none,t:lowercase,pass,nolog,ctl:requestBodyProcessor=JSON"

# Sample rule to enable JSON request body parser for more subtypes.
# Uncomment or adapt this rule if you want to engage the JSON
# Processor for "+json" subtypes
#
#SecRule REQUEST_HEADERS:Content-Type "^application/[a-z0-9.-]+[+]json" \
#     "id:'200006',phase:1,t:none,t:lowercase,pass,nolog,ctl:requestBodyProcessor=JSON"

# Maximum request body size we will accept for buffering. If you support
# file uploads then the value given on the first line has to be as large
# as the largest file you are willing to accept. The second value refers
```

Annexe 14

```
  GNU nano 7.2                    /etc/fail2ban/jail.local
[DEFAULT]
bantime = 3600
findtime = 600
maxretry = 5
ignoreip = 127.0.0.1/8 ::1
[apache-auth]
enabled = true
port = http,https
filter = apache-auth
logpath = /var/log/apache2/*error.log
maxretry = 3
[apache-badbots]
enabled = true
port = http,https
filter = apache-badbots
logpath = /var/log/apache2/*access.log
maxretry = 2
[apache-noscript]
enabled = true
port = http,https
                              [ Lecture de 34 lignes ]
^G Aide      ^O Écrire    ^W Chercher  ^K Couper    ^T Exécuter  ^C Emplacement
^X Quitter   ^R Lire fich.^\ Remplacer ^U Coller    ^J Justifier ^/ Aller ligne
```

```
brandon@brandon: /etc/modsecurity
  GNU nano 7.2      /etc/fail2ban/filter.d/apache-modsecurity.conf *
# Fail2Ban apache-modsec filter
#

[INCLUDES]

# Read common prefixes. If any customizations available -- read them from
# apache-common.local
before = apache-common.conf

[Definition]

failregex = \[.*\] \[client <HOST>\] ModSecurity:.*
ignoreregex =

# https://github.com/SpiderLabs/ModSecurity/wiki/ModSecurity-2-Data-Formats
# Author: Daniel Black
#         Sergey G. Brester aka sebres (review, optimization)



^G Aide      ^O Écrire    ^W Chercher  ^K Couper    ^T Exécuter  ^C Emplacement
^X Quitter   ^R Lire fich.^\ Remplacer ^U Coller    ^J Justifier ^/ Aller ligne
```

```
brandon@brandon: /etc/modsecurity
brandon@brandon:/etc/modsecurity$ sudo nano /etc/fail2ban/filter.d/apache-modsec
urity.conf
brandon@brandon:/etc/modsecurity$ sudo systemctl enable fail2ban
Synchronizing state of fail2ban.service with SysV service script with /usr/lib/s
ystemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable fail2ban
brandon@brandon:/etc/modsecurity$
```

```
brandon@brandon: /etc/modsecurity
brandon@brandon:/etc/modsecurity$ sudo fail2ban-client status
Status
|- Number of jail:      6
`- Jail list:   apache-auth, apache-badbots, apache-modsecurity, apache-noscript, apach
e-overflows, sshd
brandon@brandon:/etc/modsecurity$
```

```
brandon@brandon:/etc/modsecurity$ sudo fail2ban-client status apache-auth
Status for the jail: apache-auth
|- Filter
|  |- Currently failed: 0
|  |- Total failed:     0
|  `- Journal matches:
`- Actions
   |- Currently banned: 0
   |- Total banned:     0
   `- Banned IP list:
brandon@brandon:/etc/modsecurity$
```

Annexe 15

```
26 déc. 15:24
brandon@brandon: /etc/modsecurity
  GNU nano 7.2          /etc/apache2/conf-available/security.conf *
#RedirectMatch 404 /\.svn

#
# Setting this header will prevent MSIE from interpreting files as something
# else than declared by the content type in the HTTP headers.
# Requires mod_headers to be enabled.
#
#Header set X-Content-Type-Options: "nosniff"

#
# Setting this header will prevent other sites from embedding pages from this
# site as frames. This defends against clickjacking attacks.
# Requires mod_headers to be enabled.
#
#Header set Content-Security-Policy "frame-ancestors 'self';"
# Protection contre le clickjacking
Header always set X-Frame-Options "SAMEORIGIN"
# Protection XSS (navigateurs anciens)
Header always set X-XSS-Protection "1; mode=block"
# Empêcher le MIME-sniffing
Header always set X-Content-Type-Options "nosniff"
# Politique de référent
Header always set Referrer-Policy "strict-origin-when-cross-origin"
# Content Security Policy (CSP)
Header always set Content-Security-Policy "default-src 'self'; \
script-src 'self'; style-src 'self' 'unsafe-inline'; \
img-src 'self' data:; font-src 'self'; \
frame-ancestors 'self'; form-action 'self'"
# Permissions Policy (remplace Feature-Policy)
Header always set Permissions-Policy "geolocation=(), microphone=(), camera=()"
# Strict Transport Security (HSTS) - uniquement si HTTPS Header always set StrictTrans>
"max-age=31536000; includeSubDomains; preload"
```

Annexe 16

```
brandon@brandon:/etc/modsecurity$ curl 'http://localhost/?param=<script>alert(1)
</script>'
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>403 Forbidden</title>
</head><body>
<h1>Forbidden</h1>
<p>You don't have permission to access this resource.</p>
<hr>
<address>Apache/2.4.58 (Ubuntu) Server at localhost Port 80</address>
</body></html>
brandon@brandon:/etc/modsecurity$
```

```
brandon@brandon:/etc/modsecurity$ sudo tail -f /var/log/apache2/modsec_audit.log
Action: Intercepted (phase 2)
Stopwatch: 1766757648348968 3331 (- - -)
Stopwatch2: 1766757648348968 3331; combined=2778, p1=1102, p2=1528, p3=0, p4=0,
p5=148, sr=0, sw=0, l=0, gc=0
Response-Body-Transformed: Dechunked
Producer: ModSecurity for Apache/2.9.7 (http://www.modsecurity.org/); OWASP_CRS/
4.22.0-dev.
Server: Apache/2.4.58 (Ubuntu)
Engine-Mode: "ENABLED"

--dc6a4b58-Z--
```

Annexe 17

```
brandon@brandon:/etc/modsecurity$ sudo a2enmod headers
Enabling module headers.
To activate the new configuration, you need to run:
  systemctl restart apache2
brandon@brandon:/etc/modsecurity$  sudo systemctl restart apache2
brandon@brandon:/etc/modsecurity$
```

```
brandon@brandon:/etc/modsecurity$ curl -I https://localhost
curl: (7) Failed to connect to localhost port 443 after 0 ms: Couldn't connect to serve
r
brandon@brandon:/etc/modsecurity$ curl -I http://localhost
HTTP/1.1 200 OK
Date: Fri, 26 Dec 2025 14:26:40 GMT
Server: Apache/2.4.58 (Ubuntu)
X-Frame-Options: SAMEORIGIN
X-XSS-Protection: 1; mode=block
X-Content-Type-Options: nosniff
Referrer-Policy: strict-origin-when-cross-origin
Content-Security-Policy: default-src 'self'; script-src 'self'; style-src 'self' 'unsaf
e-inline'; img-src 'self' data:; font-src 'self'; frame-ancestors 'self'; form-action '
self'
Permissions-Policy: geolocation=(), microphone=(), camera=()
Last-Modified: Fri, 26 Dec 2025 13:41:38 GMT
ETag: "29af-646db0d85d3fd"
Accept-Ranges: bytes
Content-Length: 10671
Vary: Accept-Encoding
Content-Type: text/html

brandon@brandon:/etc/modsecurity$
```

```
brandon@brandon:~/Bureau$ curl -I -k https://localhost | grep -i content-security-policy
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0 10671    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
Content-Security-Policy: default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' d
brandon@brandon:~/Bureau$
```

Annexe 18

```
brandon@brandon:/etc/modsecurity$ sudo goaccess /var/log/apache2/access.log -c
Cleaning up resources...
```

```
brandon@brandon: /etc/modsecurity
Fichier  Édition  Affichage  Rechercher  Terminal  Aide

+-------------------------------------------------------------------+
| Configuration format des logs                                     |
| [SPACE] pour basculer - [ENTREE] pour procéder - [q] pour quitter |
|                                                                   |
| [x] NCSA Combined Log Format                                      |
| [ ] NCSA Combined Log Format with Virtual Host                    |
| [ ] Common Log Format (CLF)                                       |
| [ ] Common Log Format (CLF) with Virtual Host                     |
| [ ] W3C                                                           |
| [ ] CloudFront (Download Distribution)                            |
|                                                                   |
| Format des logs - [c] pour ajouter/éditer le format               |
| %h %^[%d:%t %^] "%r" %s %b "%R" "%u"                              |
|                                                                   |
| Format de la date - [d] pour ajouter/éditer le format             |
| %d/%b/%Y                                                          |
|                                                                   |
| Format temporel - [t] pour ajouter/éditer le format               |
| %H:%M:%S                                                          |
+-------------------------------------------------------------------+
```

Annexe 19

```
brandon@brandon: /etc/modsecurity
Fichier  Édition  Affichage  Rechercher  Terminal  Aide
Tableau de bord - Requêtes analysées, vue d'ensemble (26/dé[Panneau Actif: Visiteurs]

  Requêtes totales      6 Visiteurs uniques 1  Requested Files     2 Origine      0
  Requêtes valides      3 Log Parsing Time  1s Fichiers statiques 0 Taille log   302.0
B Requêtes échouées     0 Excl. IP Hits     0  Non Trouvé          0 Tx. Montant 1.48 KiB
  Fichier log           /var/log/apache2/access.log

> 1 - Visiteurs uniques/jour - Y compris bots                          Total: 1/1

 Hits      h% Vis.      v% Tx. Montant Données
 ---- ------- ---- ------- ----------- ----
    3 100.00%    1 100.00%    1.48 KiB 26/déc./2025  ||||||||||||||||||||||||||||||||||

  2 - Fichiers demandés (URLs)                                           Total: 2/2

 Hits     h% Vis.      v% Tx. Montant Mtd  Proto    Données
 ---- ------ ---- ------- ----------- ---- -------- ----
    2 66.67%    0   0.00%    870.0    B GET  HTTP/1.1 /?param=<script>alert(1)</script>
    1 33.33%    1 100.00%    643.0    B HEAD HTTP/1.1 /

  3 - Requêtes statiques                                                 Total: 0/0

 [?] Aide [Enter] Dév. Panneau   0/r - 26/déc./2025:15:50:10   [q]uitter GoAccess 1.8.1
```

```
brandon@brandon:~$ sudo goaccess /var/log/apache2/access.log -o /var/www/html/re
port.html --log-format=COMBINED
 [PARSING /var/log/apache2/access.log] {0} @ {0/s}
Cleaning up resources...
brandon@brandon:~$
```

```
brandon@brandon: ~
brandon@brandon:~$ sudo goaccess /var/log/apache2/access.log -o /var/www/html/re
port.html --log-format=COMBINED --real-time-html
 [Parsing... /var/log/apache2/access.log] {0} @ {0/s}
WebSocket serveur prêt à accepter les nouvelles connexions client
```

Annexe 20

```
brandon@brandon:~$ sudo goaccess /var/log/apache2/access.log -o /var/www/html/report.html --log-format=COMBINED --real-time-html
 [Parsing... /var/log/apache2/access.log] {0} @ {0/s}
WebSocket serveur prêt à accepter les nouvelles connexions client
```

```
  GNU nano 7.2                    /etc/logrotate.d/apache2 *
/var/log/apache2/*.log { daily
Missingok
rotate 14
compress
delaycompress
notifempty
create 640 root adm
sharedscripts
postrotate
/etc/init.d/apache2 reload > /dev/null
endscript
}

^G Aide      ^O Écrire      ^W Chercher  ^K Couper    ^T Exécuter  ^C Emplacement
^X Quitter   ^R Lire fich.  ^\ Remplacer ^U Coller    ^J Justifier ^/ Aller ligne
```

Annexe 21

```
brandon@brandon:~$ nikto -h https://localhost -ssl
```

```
brandon@brandon: ~
brandon@brandon:~$ nikto -h https://localhost -ssl
- Nikto v2.1.5
---------------------------------------------------------------------------
+ Target IP:          127.0.0.1
+ Target Hostname:    localhost
+ Target Port:        443
---------------------------------------------------------------------------
+ SSL Info:        Subject: /C=FR/ST=Paris/L=Chantilly/O=Efrei/OU=Security/CN=lo
calhost
                   Ciphers: TLS_AES_256_GCM_SHA384
                   Issuer:  /C=FR/ST=Paris/L=Chantilly/O=Efrei/OU=Security/CN=lo
calhost
+ Start Time:         2025-12-26 15:57:55 (GMT1)
---------------------------------------------------------------------------
+ Server: Apache/2.4.58 (Ubuntu)
+ Uncommon header 'x-xss-protection' found, with contents: 1; mode=block
+ Uncommon header 'x-content-type-options' found, with contents: nosniff
+ Uncommon header 'permissions-policy' found, with contents: geolocation=(), mic
rophone=(), camera=()
+ Uncommon header 'content-security-policy' found, with contents: default-src 's
elf'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data:;
 font-src 'self'; frame-ancestors 'self'; form-action 'self'
+ Uncommon header 'referrer-policy' found, with contents: strict-origin-when-cro
ss-origin
```

```
brandon@brandon: ~
brandon@brandon:~$  git clone --depth 1 https://github.com/drwetter/testssl.sh.g
it
Clonage dans 'testssl.sh'...
remote: Enumerating objects: 104, done.
remote: Counting objects: 100% (104/104), done.
remote: Compressing objects: 100% (97/97), done.
remote: Total 104 (delta 12), reused 41 (delta 7), pack-reused 0 (from 0)
Réception d'objets: 100% (104/104), 6.33 Mio | 11.39 Mio/s, fait.
Résolution des deltas: 100% (12/12), fait.
brandon@brandon:~$
```

Annexe 22

```
brandon@brandon: ~/testssl.sh
brandon@brandon:~/testssl.sh$ ./testssl.sh https://localhost
```

```
brandon@brandon: ~/testssl.sh

 Testing protocols via sockets except NPN+ALPN

 SSLv2      not offered (OK)
 SSLv3      not offered (OK)
 TLS 1      not offered
 TLS 1.1    not offered
 TLS 1.2    offered (OK)
 TLS 1.3    offered (OK): final
 QUIC       Local problem: No OpenSSL QUIC support
 NPN/SPDY   not offered
 ALPN/HTTP2 http/1.1 (offered)

 Testing cipher categories

 NULL ciphers (no encryption)                      not offered (OK)
 Anonymous NULL Ciphers (no authentication)        not offered (OK)
 Export ciphers (w/o ADH+NULL)                     not offered (OK)
 LOW: 64 Bit + DES, RC[2,4], MD5 (w/o export)      not offered (OK)
 Triple DES Ciphers / IDEA                         not offered
 Obsoleted CBC ciphers (AES, ARIA etc.)            offered
 Strong encryption (AEAD ciphers) with no FS       offered (OK)
 Forward Secrecy strong encryption (AEAD ciphers)  offered (OK)
```

```
 Testing vulnerabilities

 Heartbleed (CVE-2014-0160)                not vulnerable (OK), no heartbeat ext
ension
 CCS (CVE-2014-0224)                       not vulnerable (OK)
 Ticketbleed (CVE-2016-9244), experiment.  not vulnerable (OK), no session ticke
t extension
 Opossum (CVE-2025-49812)                  not vulnerable (OK)
 ROBOT                                     not vulnerable (OK)
 Secure Renegotiation (RFC 5746)           supported (OK)
 Secure Client-Initiated Renegotiation     not vulnerable (OK)
 CRIME, TLS (CVE-2012-4929)                not vulnerable (OK)
 BREACH (CVE-2013-3587)                    potentially NOT ok, "gzip" HTTP compr
ession detected. - only supplied "/" tested
                                           Can be ignored for static pages or if
 no secrets in the page
 POODLE, SSL (CVE-2014-3566)               not vulnerable (OK), no SSLv3 support
 TLS_FALLBACK_SCSV (RFC 7507)              No fallback possible (OK), no protoco
l below TLS 1.2 offered
 SWEET32 (CVE-2016-2183, CVE-2016-6329)
```

Annexe 23

Running client simulations (HTTP) via sockets

Browser	Protocol	Cipher Suite Name (OpenSSL)	Forward Secrecy
Android 7.0 (native)	TLSv1.2	ECDHE-RSA-AES128-GCM-SHA256	256 bit ECDH (P-256)
Android 8.1 (native)	TLSv1.2	ECDHE-RSA-AES128-GCM-SHA256	253 bit ECDH (X25519)
Android 9.0 (native)	TLSv1.3	TLS_AES_128_GCM_SHA256	253 bit ECDH (X25519)
Android 10.0 (native)	TLSv1.3	TLS_AES_128_GCM_SHA256	253 bit ECDH (X25519)
Android 11/12 (native)	TLSv1.3	TLS_AES_128_GCM_SHA256	253 bit ECDH (X25519)
Android 13/14 (native)	TLSv1.3	TLS_AES_128_GCM_SHA256	253 bit ECDH (X25519)
Android 15 (native)	TLSv1.3	TLS_AES_128_GCM_SHA256	253 bit ECDH (X25519)
Chrome 101 (Win 10)	TLSv1.3	TLS_AES_128_GCM_SHA256	253 bit ECDH (X25519)
Chromium 137 (Win 11)	TLSv1.3	TLS_AES_128_GCM_SHA256	253 bit ECDH (X25519)
Firefox 100 (Win 10)	TLSv1.3	TLS_AES_128_GCM_SHA256	253 bit ECDH (X25519)
Firefox 137 (Win 11)	TLSv1.3	TLS_AES_128_GCM_SHA256	253 bit ECDH (X25519)
IE 8 Win 7	No connection		
IE 11 Win 7	TLSv1.2	ECDHE-RSA-AES256-SHA384	256 bit ECDH (P-256)
IE 11 Win 8.1	TLSv1.2	ECDHE-RSA-AES256-SHA384	256 bit ECDH (P-256)
IE 11 Win Phone 8.1	TLSv1.2	AES128-SHA256	No FS
IE 11 Win 10	TLSv1.2	ECDHE-RSA-AES256-GCM-SHA384	256 bit ECDH (P-256)
Edge 15 Win 10	TLSv1.2	ECDHE-RSA-AES256-GCM-SHA384	253 bit ECDH (X25519)
Edge 101 Win 10 21H2	TLSv1.3	TLS_AES_128_GCM_SHA256	253 bit ECDH (X25519)
Edge 133 Win 11 23H2	TLSv1.3	TLS_AES_128_GCM_SHA256	253 bit ECDH (X25519)
Safari 18.4 (iOS 18.4)	TLSv1.3	TLS_AES_128_GCM_SHA256	253 bit ECDH (X25519)
Safari 15.4 (macOS 12.3.1)	TLSv1.3	TLS_AES_128_GCM_SHA256	253 bit ECDH (X25519)
Safari 18.4 (macOS 15.4)	TLSv1.3	TLS_AES_128_GCM_SHA256	253 bit ECDH (X25519)
Java 7u25	No connection		
Java 8u442 (OpenJDK)	TLSv1.3	TLS_AES_256_GCM_SHA384	253 bit ECDH (X25519)
Java 11.0.2 (OpenJDK)	TLSv1.3	TLS_AES_128_GCM_SHA256	256 bit ECDH (P-256)
Java 17.0.3 (OpenJDK)	TLSv1.3	TLS_AES_256_GCM_SHA384	253 bit ECDH (X25519)
Java 21.0.6 (OpenJDK)	TLSv1.3	TLS_AES_256_GCM_SHA384	253 bit ECDH (X25519)
go 1.17.8	TLSv1.3	TLS_AES_128_GCM_SHA256	253 bit ECDH (X25519)
LibreSSL 3.3.6 (macOS)	TLSv1.3	TLS_CHACHA20_POLY1305_SHA256	253 bit ECDH (X25519)
OpenSSL 1.0.2e	TLSv1.2	ECDHE-RSA-AES256-GCM-SHA384	256 bit ECDH (P-256)
OpenSSL 1.1.1d (Debian)	TLSv1.3	TLS_AES_256_GCM_SHA384	253 bit ECDH (X25519)
OpenSSL 3.0.15 (Debian)	TLSv1.3	TLS_AES_256_GCM_SHA384	253 bit ECDH (X25519)
OpenSSL 3.5.0 (git)	TLSv1.3	TLS_AES_256_GCM_SHA384	253 bit ECDH (X25519)
Apple Mail (16.0)	TLSv1.2	ECDHE-RSA-AES256-GCM-SHA384	256 bit ECDH (P-256)
Thunderbird (91.9)	TLSv1.3	TLS_AES_128_GCM_SHA256	253 bit ECDH (X25519)

```
brandon@brandon:~/testssl.sh$ ./testssl.sh --html https://localhost

###########################################################################
  testssl.sh version 3.3dev from https://testssl.sh/dev/
  (6a5a69f 2025-12-20 23:24:50)

  This program is free software. Distribution and modification under
  GPLv2 permitted. USAGE w/o ANY WARRANTY. USE IT AT YOUR OWN RISK!

  Please file bugs @ https://testssl.sh/bugs/
###########################################################################

  Using OpenSSL 1.0.2-bad (Mar 28 2025)  [~179 ciphers]
  on brandon:./bin/openssl.Linux.x86_64

Testing all IPv4 addresses (port 443): 127.0.0.1
---------------------------------------------------------
 Start 2025-12-26 16:04:11        -->> 127.0.0.1:443 (localhost) <<--

 A record via:           /etc/hosts
 rDNS (127.0.0.1):       localhost.
 Service detected:       HTTP

 Testing protocols via sockets except NPN+ALPN

 SSLv2      not offered (OK)
 SSLv3      not offered (OK)
 TLS 1      not offered
 TLS 1.1    not offered
 TLS 1.2    offered (OK)
```

```
TLSv1
 -
TLSv1.1
 -
TLSv1.2 (no server order, thus listed by strength)
 xc030   ECDHE-RSA-AES256-GCM-SHA384       ECDH 521   AESGCM      256      TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384
 xc028   ECDHE-RSA-AES256-SHA384           ECDH 521   AES         256      TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384
 xc014   ECDHE-RSA-AES256-SHA              ECDH 521   AES         256      TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA
 x9f     DHE-RSA-AES256-GCM-SHA384         DH 2048    AESGCM      256      TLS_DHE_RSA_WITH_AES_256_GCM_SHA384
 xcca8   ECDHE-RSA-CHACHA20-POLY1305       ECDH 521   ChaCha20    256      TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256
 xccaa   DHE-RSA-CHACHA20-POLY1305         DH 2048    ChaCha20    256      TLS_DHE_RSA_WITH_CHACHA20_POLY1305_SHA256
 xc0a3   DHE-RSA-AES256-CCM8               DH 2048    AESCCM8     256      TLS_DHE_RSA_WITH_AES_256_CCM_8
 xc09f   DHE-RSA-AES256-CCM                DH 2048    AESCCM      256      TLS_DHE_RSA_WITH_AES_256_CCM
 x6b     DHE-RSA-AES256-SHA256             DH 2048    AES         256      TLS_DHE_RSA_WITH_AES_256_CBC_SHA256
 x39     DHE-RSA-AES256-SHA                DH 2048    AES         256      TLS_DHE_RSA_WITH_AES_256_CBC_SHA
 xc077   ECDHE-RSA-CAMELLIA256-SHA384      ECDH 521   Camellia    256      TLS_ECDHE_RSA_WITH_CAMELLIA_256_CBC_SHA384
 xc4     DHE-RSA-CAMELLIA256-SHA256        DH 2048    Camellia    256      TLS_DHE_RSA_WITH_CAMELLIA_256_CBC_SHA256
 x88     DHE-RSA-CAMELLIA256-SHA           DH 2048    Camellia    256      TLS_DHE_RSA_WITH_CAMELLIA_256_CBC_SHA
 x9d     AES256-GCM-SHA384                 RSA        AESGCM      256      TLS_RSA_WITH_AES_256_GCM_SHA384
 xc0a1   AES256-CCM8                       RSA        AESCCM8     256      TLS_RSA_WITH_AES_256_CCM_8
 xc09d   AES256-CCM                        RSA        AESCCM      256      TLS_RSA_WITH_AES_256_CCM
 x3d     AES256-SHA256                     RSA        AES         256      TLS_RSA_WITH_AES_256_CBC_SHA256
 x35     AES256-SHA                        RSA        AES         256      TLS_RSA_WITH_AES_256_CBC_SHA
 xc0     CAMELLIA256-SHA256                RSA        Camellia    256      TLS_RSA_WITH_CAMELLIA_256_CBC_SHA256
 x84     CAMELLIA256-SHA                   RSA        Camellia    256      TLS_RSA_WITH_CAMELLIA_256_CBC_SHA
 xc051   ARIA256-GCM-SHA384                RSA        ARIAGCM     256      TLS_RSA_WITH_ARIA_256_GCM_SHA384
 xc053   DHE-RSA-ARIA256-GCM-SHA384        DH 2048    ARIAGCM     256      TLS_DHE_RSA_WITH_ARIA_256_GCM_SHA384
 xc061   ECDHE-ARIA256-GCM-SHA384          ECDH 521   ARIAGCM     256      TLS_ECDHE_RSA_WITH_ARIA_256_GCM_SHA384
 xc02f   ECDHE-RSA-AES128-GCM-SHA256       ECDH 521   AESGCM      128      TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256
 xc027   ECDHE-RSA-AES128-SHA256           ECDH 521   AES         128      TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256
 xc013   ECDHE-RSA-AES128-SHA              ECDH 521   AES         128      TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA
 x9e     DHE-RSA-AES128-GCM-SHA256         DH 2048    AESGCM      128      TLS_DHE_RSA_WITH_AES_128_GCM_SHA256
 xc0a2   DHE-RSA-AES128-CCM8               DH 2048    AESCCM8     128      TLS_DHE_RSA_WITH_AES_128_CCM_8
 xc09e   DHE-RSA-AES128-CCM                DH 2048    AESCCM      128      TLS_DHE_RSA_WITH_AES_128_CCM
 xc0a0   AES128-CCM8                       RSA        AESCCM8     128      TLS_RSA_WITH_AES_128_CCM_8
 xc09c   AES128-CCM                        RSA        AESCCM      128      TLS_RSA_WITH_AES_128_CCM
 x67     DHE-RSA-AES128-SHA256             DH 2048    AES         128      TLS_DHE_RSA_WITH_AES_128_CBC_SHA256
 x33     DHE-RSA-AES128-SHA                DH 2048    AES         128      TLS_DHE_RSA_WITH_AES_128_CBC_SHA
 xc076   ECDHE-RSA-CAMELLIA128-SHA256      ECDH 521   Camellia    128      TLS_ECDHE_RSA_WITH_CAMELLIA_128_CBC_SHA256
 xbe     DHE-RSA-CAMELLIA128-SHA256        DH 2048    Camellia    128      TLS_DHE_RSA_WITH_CAMELLIA_128_CBC_SHA256
 x45     DHE-RSA-CAMELLIA128-SHA           DH 2048    Camellia    128      TLS_DHE_RSA_WITH_CAMELLIA_128_CBC_SHA
 x9c     AES128-GCM-SHA256                 RSA        AESGCM      128      TLS_RSA_WITH_AES_128_GCM_SHA256
 x3c     AES128-SHA256                     RSA        AES         128      TLS_RSA_WITH_AES_128_CBC_SHA256
 x2f     AES128-SHA                        RSA        AES         128      TLS_RSA_WITH_AES_128_CBC_SHA
 xba     CAMELLIA128-SHA256                RSA        Camellia    128      TLS_RSA_WITH_CAMELLIA_128_CBC_SHA256
```

```
brandon@brandon:~/testssl.sh$ curl -I -k https://localhost
HTTP/1.1 200 OK
Date: Fri, 26 Dec 2025 15:05:28 GMT
Server: Apache/2.4.58 (Ubuntu)
X-Frame-Options: SAMEORIGIN
X-XSS-Protection: 1; mode=block
X-Content-Type-Options: nosniff
Referrer-Policy: strict-origin-when-cross-origin
Content-Security-Policy: default-src 'self'; script-src 'self'; style-src 'self' 'unsaf
e-inline'; img-src 'self' data:; font-src 'self'; frame-ancestors 'self'; form-action '
self'
Permissions-Policy: geolocation=(), microphone=(), camera=()
Last-Modified: Fri, 26 Dec 2025 13:41:38 GMT
ETag: "29af-646db0d85d3fd"
Accept-Ranges: bytes
Content-Length: 10671
Vary: Accept-Encoding
Content-Type: text/html

brandon@brandon:~/testssl.sh$
```

Annexe 24

```
brandon@brandon: ~/testssl.sh
brandon@brandon:~/testssl.sh$ curl -I -k https://localhost | grep -i "content-security-
policy"
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0 10671    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
Content-Security-Policy: default-src 'self'; script-src 'self'; style-src 'self' 'unsaf
e-inline'; img-src 'self' data:; font-src 'self'; frame-ancestors 'self'; form-action '
self'
brandon@brandon:~/testssl.sh$
```

```
brandon@brandon:~/testssl.sh$ curl -v -k https://localhost 2>&1 | grep -E "^(<|>)"
> GET / HTTP/1.1
> Host: localhost
> User-Agent: curl/8.5.0
> Accept: */*
>
< HTTP/1.1 200 OK
< Date: Fri, 26 Dec 2025 15:06:23 GMT
< Server: Apache/2.4.58 (Ubuntu)
< X-Frame-Options: SAMEORIGIN
< X-XSS-Protection: 1; mode=block
< X-Content-Type-Options: nosniff
< Referrer-Policy: strict-origin-when-cross-origin
< Content-Security-Policy: default-src 'self'; script-src 'self'; style-src 'self' 'uns
afe-inline'; img-src 'self' data:; font-src 'self'; frame-ancestors 'self'; form-action
 'self'
< Permissions-Policy: geolocation=(), microphone=(), camera=()
< Last-Modified: Fri, 26 Dec 2025 13:41:38 GMT
< ETag: "29af-646db0d85d3fd"
< Accept-Ranges: bytes
< Content-Length: 10671
< Vary: Accept-Encoding
< Content-Type: text/html
<
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xh
tml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
</html>
brandon@brandon:~/testssl.sh$
```